Trellis Code Review Request

https://gemini.google.com/app/e50824e7fca8945b

User prompt: Can you review the implementations at C:\Work\Lapwing\Trellis and suggest performance enhancements modularization, simplification. Create an implementation plane at C:\Work\Lapwing\Trellis\Wiki

Response: I cannot access your local filesystem (`C:\Work\Lapwing\Trellis`) directly from this environment, and the `Trellis` item attached in the notebook did not pass through any readable source files.

To get started, share the code using whichever option is easiest for you:

1. **Paste the key files directly** into the chat (such as your core modules, `Cargo.toml` / `CMakeLists.txt` / build config, and hot-path implementations).
2. **Dump the repository into a single text file** and upload or paste it here. You can run this in PowerShell from `C:\Work\Lapwing\Trellis` to bundle your source files:

```
Get-ChildItem -Path . -Recurse -Include *.rs,*.cpp,*.hpp,*.h,*.c,*.mojo,*.py,*.toml,*.cmake |
  Where-Object { $_.FullName -notmatch '\\(target|build|\.git)\\' } |
  ForEach-Object { "=== FILE: $($_.FullName) ==="; Get-Content $_.FullName; "" } |
  Out-File -FilePath .\trellis_dump.txt -Encoding utf8
```

Once you share the code, I will review the architecture for performance bottlenecks, modularization, and simplification, and provide the complete Markdown implementation plan ready to drop into `C:\Work\Lapwing\Trellis\Wiki`.

User prompt: Here

Response: 1. Executive Review: Performance, Modularization & Simplification

(Note: `trellis_dump.txt` was truncated by the attachment size limit inside `src\fascia\_tests.rs`. The review below covers `examples/rube_perf.rs`, `cove`, `crew`, `drove`, and `fascia` in full, along with their integration boundaries into `rube`, `silo`, `stalks`, `heist`, `fenst`, `fleck`, `swarm`, and `symph`. See the end of this response for a one-liner to dump the remaining core modules without test files.)

A. Critical Performance Enhancements

1. `fascia::menubar` — Per-Frame Heap Memory Leak (`src/fascia/menubar.rs`)

- **Issue**: Inside `view_menubar`, the theme menu loop formats a `String` for each of the 5 `FasciaTheme` variants and calls `Box::leak(label_owned.into_boxed_str())` to obtain a `'static str`. Because Iced invokes `AppState::view()` on every mouse move, scroll, and redraw event, this permanently leaks 5 heap strings per rendered frame.
- **Fix**: Replace dynamic string formatting with a `const fn menu_label(&self, is_active: bool) -> &'static str` on `FasciaTheme` returning static string literals.

2. `crew::hub` — Per-Transaction Heap Allocations, Linear Scans & Lock Contention (`src/crew/hub.rs`)

- **Issue**: `CrewHub::HandleRequest` calls `self.getPeersForNode(node.Id())` on every `REG_STATUS` read and every `REG_TX_DATA` byte write.
  - `getPeersForNode` acquires `self._Routes.Lock()`, performs a non-short-circuiting `USeg::Traverse` across all routes, and clones `Buff<u32>` (a heap allocation) on every register operation.
  - If no explicit route exists, it calls `self.FindNode(peerId)` *while still holding* `_Routes.Lock()` (a nested SpinMutex acquisition) and heap-allocates a 1-element `Buff` via `Buff::FromDispenser`.
  - In `WriteBusByte`, `HandleRequest` calls `FindNode(peerId)` (which locks `_Nodes`, scans all nodes without early exit, and clones `Arc<CrewNode>`) **twice per peer**—once to check `CanPushRx()` and a second time to call `TryPushRx(byte)`—creating both redundant lock traffic and a TOCTOU race between the status check and the push.
  - Inside the peer delivery loop, `self._MessageCb.Lock()` is also acquired on every iteration rather than once before the loop.
- **Fix**:
  - Store resolved peer `Buff<Arc<CrewNode>>` directly inside each `CrewNode` (updated only during `AddNode`/`AddLink` configuration) or use `USeg::Span` for early exit.
  - Eliminate `getPeersForNode` heap allocations from `HandleRequest` entirely.
  - Read `_MessageCb` once outside the delivery loop.

3. `crew::vm_adaptor` & `crew::vm_runner` — Per-Cycle `SpinMutex` & `CoroPorts` Overhead (`src/crew/vm_adaptor.rs`, `src/crew/vm_runner.rs`)

- **Issue**:
  - `VMAdaptor` stores `_Stats: Arc<SpinMutex<NodeStats>>` and locks the `SpinMutex` inside the coroutine loop on every transmitted byte, received byte, MMIO read, and MMIO write. Meanwhile, `CrewNode` already tracks the exact same counters using lock-free `AtomicU32` fields with `Ordering::Relaxed`.

- Every cycle of `VMAdaptor` and `VmBus::Idle/Read/Write` builds a `CoroPorts::New()` and calls `.Push(...)` 4 to 5 times.

- **Fix**:
  - Extract a shared lock-free `AtomicNodeStats` struct (6 `AtomicU32` counters) used by both `CrewNode` and `VMAdaptor`, removing `SpinMutex<NodeStats>` from the simulation hot loop.
  - Add fixed-size array constructors (e.g., `CoroPorts::FromArray`) so `VmBus::Idle()`, `VmBus::Read()`, `VmBus::Write()`, and `VMAdaptor` yield ports in a single stack write without repeated `.Push()` calls.

4. `fascia::cask_labels` & `fascia::cask_scene` — Triple HarfBuzz/Cosmic-Text Shaping (`src/fascia/cask_labels.rs`, `src/fascia/cask_scene.rs`)

- **Issue**: When building a 3D Cask scene in `Build` or `BuildWithViewBox`:
  1. `LabelPlan::LayoutScene` runs `text.set_text(..., Shaping::Advanced, ...)` on **every node** in the scene.
  2. `scene.Unfurl(&viewbox, cancelled)` filters the scene to visible nodes.
  3. `LabelPlan::PlanForUnfurled` runs `text.set_text(..., Shaping::Advanced, ...)` a **second time** on every unfurled node to re-compute its `[f32; 2]` size.
  4. `LabelPlan::Raster` runs `text.set_text(..., Shaping::Advanced, ...)` a **third time** on every unfurled node to rasterize glyphs into the atlas.

- **Fix**:
  - Persist the measured label `[f32; 2]` dimensions on `CaskNode` during `LayoutScene` so `PlanForUnfurled` reads cached sizes in $O(1)$ without re-locking `FONTS` or re-shaping text.
  - Select `Shaping::Basic` when `name.is_ascii()` and `Shaping::Advanced` only for non-ASCII strings, cutting text measurement time by 5–10x on source trees and module hierarchies.

5. `fascia::waveform` — Sub-Pixel Overdraw on Zoomed-Out VCD Traces (`src/fascia/waveform.rs`)

- **Issue**: `view_waveform` instantiates up to 64 `canvas::Program` (`WaveformLane`) instances. None use `canvas::Cache`, and `WaveformLane::draw` strokes two `Path::line` segments for every signal transition between `_ViewStart` and `_ViewEnd`. On a trace with 100k transitions viewed in `Fit` mode across a 1,000px viewport, this issues 200k line strokes per lane on every redraw, drawing ratusan of overlapping vertical lines on the exact same pixel column.

- **Fix**:
  - Implement pixel-column envelope decimation inside `WaveformLane::draw`: quantize `x` to integer pixel columns (`x.floor() as i32`) and coalesce multiple transitions within the same column into a single vertical min-to-max bar.
  - Back each lane (or a unified multi-lane waveform canvas) with `canvas::Cache`, invalidating only on `ViewStart`/`ViewEnd` or resize changes.

6. `fascia::cask_view` — Uncached Dot Grid, Per-Frame Lowercasing & Hit-Test Order (`src/fascia/cask_view.rs`)

- **Issue**:
  - `Cask2dCanvasProgram::draw` redraws the entire infinite dot grid via nested `while` loops (`frame.fill_rectangle` per dot) and clones every label `String` on every mouse cursor movement.
  - When `search_query` is non-empty, `draw` calls `node.Name().to_lowercase().contains(&query)` for every node on every frame, allocating a new `String` per node per frame.
  - In `Cask2dCanvasProgram::update`, left-click hit-testing uses `l.Nodes().iter().position(|n| n.ContainsPoint(world_x, world_y))`. In `NestedBoxes` mode, the root node is at index `0` and encloses all child bounds, so `.position(...)` always matches the outermost parent instead of the clicked child.

- **Fix**:
  - Separate static geometry (dot grid + layout commands) into a `canvas::Cache` that only clears on `Recompute`, `Zoom`, `Pan`, or `Resize`.
  - Precompute search match indices on `Cask2dAction::Search(query)` and `Recompute()` instead of inside `draw()`.
  - Use `.iter().rposition(...)` (deepest/last-drawn node first) for point hit-testing.

7. `fascia::geometry_load` & `fascia::examples` — Per-Job `Atelier` Construction & Synchronous File I/O (`src/fascia/geometry_load.rs`, `src/fascia/examples.rs`, `src/fascia/app.rs`)

- **Issue**:
  - `geometry_load::Queue()` spawns a single background thread that constructs and launches a brand-new `Atelier::New(1)` thread pool for **every individual `LoadJob`** sequentially. Similarly, `examples::Execute` spawns a `std::thread` which then constructs an `Atelier::New(1)` inside it.
  - Opening `.vcd` files (`ParseVcd`), text files (`std::fs::read_to_string`), and computing `Cask2d` filesystem layouts (`Cask2dLayout::FromUri`) all run synchronously on the Iced GUI thread inside `AppState::update`.

- `ExplorerState` instantiates `XplrRegistry::New()` repeatedly inside `load_children()`, `root_filesystem()`, `root()`, `ensure_examples()`, and `ensure_settings()`.

- **Fix**:
  - Create a shared background `Atelier` service in `fascia` (reusing worker threads across geometry loads, VCD parsing, Cask2D layout, and example runs).
  - Share a single `XplrRegistry` instance across `ExplorerState`.

B. Modularization & Simplification Opportunities

1. **Unify Tab Document State in `fascia` (`src/fascia/tabs.rs` & `src/fascia/app.rs`)**:
   - Currently, `AppState` maintains `tab_manager: TabManager` alongside 5 separate maps (`open_editors`, `open_waveforms`, `open_consoles`, `open_cask2d_views`, and `_GeometryViews`).
   - This fragmentation already caused a bug in `MenuAction::CloseAllTabs` (`src/fascia/app.rs:141-147`), which clears 4 of the maps but forgets to cancel and clear `open_consoles` (unlike `TabBarAction::CloseAll` at lines 302–310).
   - **Simplification**: Replace the 5 maps with a single `TabDocument` enum owned directly by `TabItem` (or a single `BTreeMap<TabId, TabDocument>`). Closing a tab or closing all tabs automatically drops the document state, triggering `Drop` on `GeometryViewerState` and cancelling `PythonConsoleState` in one place.

2. **Eliminate ~40% Dual-Casing Boilerplate in `src/crew` (`hub.rs`, `node.rs`, `protocol.rs`)**:
   - `CrewHub`, `CrewNode`, and `ProtocolMessage` define both `PascalCase` and `snake_case` versions of every method (`AddNode`/`add_node`, `HandleRequest`/`handle_request`, `IsOnline`/`is_online`, `ToLeBytes`/`to_le_bytes`, etc.).
   - Standardizing on Trellis's `PascalCase` API convention removes ~200 lines of pure wrapper noise.

3. **Remove `packed` from `ProtocolMessage` (`src/crew/protocol.rs`)**:
   - `ProtocolMessage` is marked `#[repr(C, packed)]` with public fields `_ActionId: i32`, `_Addr: u64`, `_Value: u64`, `_PeripheralIndex: i32`. Because `_Addr` is at byte offset 4 and `_Value` is at offset 12, direct field access performs unaligned 64-bit loads, and taking `&msg._Addr` is undefined behavior in Rust.
   - Since wire serialization already goes through `ToLeBytes(&self) -> [u8; 24]` and `FromLeBytes(&[u8; 24])`, use standard aligned `#[repr(C)]` in memory and keep `WIRE_SIZE = 24` for `ToLeBytes`/`FromLeBytes`.

4. **Fix Hardcoded Editor Cursor Position (`src/fascia/app.rs:319`)**:
   - In `AppMessage::EditorAction`, `let (line, col) = (0, 0);` is hardcoded after `editor.perform(action)`. Replace with `let (line, col) = editor.cursor_position();` so the status bar accurately reflects cursor movement.

5. **Honor Optional `$msg` in `jeeves_assert!` Macros & Pre-Lowercase Filter (`src/cove/jeeves.rs`, `src/cove/runner.rs`)**:
   - `jeeves_assert!`, `jeeves_assert_eq!`, and `jeeves_assert_ne!` accept `$(, $msg:expr)?` but never print it on failure. Include it in the failure output.
   - In `cove::runner::run_all`, check `kind_matches` before formatting `full_name` and pre-lowercase `filter_str` once outside the `inventory::iter::<TestCase>` loop.

2. Implementation Plan for `C:\Work\Lapwing\Trellis\Wiki`

Run the PowerShell block below to create `C:\Work\Lapwing\Trellis\Wiki\Implementation_Plan.md` directly, or copy the Markdown content inside it:

```
New-Item -ItemType Directory -Force -Path "C:\Work\Lapwing\Trellis\Wiki" | Out-Null
@'
# Trellis Architecture Review & Implementation Plan

## 1. Executive Summary & Objectives

This implementation plan outlines concrete performance enhancements, modularization boundaries, and code simplificat

### Primary Targets
1. **Zero-Allocation Hot Paths**: Eliminate heap allocations, redundant `SpinMutex` acquisitions, and linear scans i
2. **UI Responsiveness & Frame Budgeting**: Fix the per-frame `Box::leak` in `menubar.rs`, add `canvas::Cache` and p
3. **State & Module Simplification**: Consolidate `AppState`'s 5 disjoint tab HashMaps/BTreeMaps into a unified `Tab

---

## 2. Prioritized Findings & Action Matrix

| ID | Module / File | Category | Issue Summary | Target Impact |
| :--- | :--- | :--- | :--- | :--- |
| **P0-1** | `src/fascia/menubar.rs` | Memory Leak | `Box::leak` called 5 times per `view()` pass for theme menu lab
| **P0-2** | `src/crew/hub.rs` | Hot-Path Perf | `getPeersForNode` locks `_Routes`, scans all entries via `Traverse`
| **P0-3** | `src/crew/vm_adaptor.rs` | Hot-Path Perf | `_Stats: Arc<SpinMutex<NodeStats>>` locked every simulation
| **P1-1** | `src/fascia/cask_labels.rs` | Load Perf | Node names shaped 3 separate times with `Shaping::Advanced` (
| **P1-2** | `src/fascia/waveform.rs` | Render Perf | 64 uncached canvases stroking $2N$ lines for $N$ transitions e
```

```
| **P1-3** | `src/fascia/cask_view.rs` | Render & UX | Uncached dot grid & text clones per frame; per-frame `.to_low
| **P1-4** | `src/fascia/geometry_load.rs` | Concurrency | New `Atelier::New(1)` created per job on a single sequent
| **P2-1** | `src/fascia/app.rs` & `tabs.rs` | Simplification | 5 separate maps for open tabs (`open_editors`, `open
| **P2-2** | `src/crew/{hub,node,protocol}.rs` | Simplification | ~200 lines of duplicate `snake_case` forwarding wr
| **P2-3** | `src/cove/{jeeves,runner}.rs` | Quality | `jeeves_assert!` ignores optional `$msg`; `run_all` formats `

---

## 3. Detailed Phased Implementation Plan

### Phase 1: Hot-Path Performance & Memory Fixes (`crew` & `fascia`)

#### Step 1.1: Eliminate `Box::leak` in `src/fascia/menubar.rs`
* Add a `const fn menu_label(&self, active: bool) -> &'static str` to `FasciaTheme` in `src/fascia/theme.rs`:
  ```rust
  pub const fn menu_label(&self, active: bool) -> &'static str {
      match (self, active) {
          (Self::WindowsDark, true)   => "✓ Windows 11 Dark (Fluent)",
          (Self::WindowsDark, false)  => "   Windows 11 Dark (Fluent)",
          (Self::WindowsLight, true)  => "✓ Windows 11 Light (Fluent)",
          (Self::WindowsLight, false) => "   Windows 11 Light (Fluent)",
          (Self::LinuxDark, true)     => "✓ Linux Dark (GNOME Adwaita)",
          (Self::LinuxDark, false)    => "   Linux Dark (GNOME Adwaita)",
          (Self::LinuxLight, true)    => "✓ Linux Light (GNOME Adwaita)",
          (Self::LinuxLight, false)   => "   Linux Light (GNOME Adwaita)",
          (Self::VsCodeDark, true)    => "✓ VS Code Dark",
          (Self::VsCodeDark, false)   => "   VS Code Dark",
      }
  }
```

- Replace lines 168–176 of `src/fascia/menubar.rs` with `theme_variant.menu_label(theme_variant == active_theme)`.

Step 1.2: Zero-Allocation Routing & Lock-Free Stats in `src/crew`

- **Unify Atomic Stats (`src/crew/node.rs` & `src/crew/vm_adaptor.rs`)**:
  - Extract `AtomicNodeStats` containing the 6 `AtomicU32` counters (`_BytesSent`, `_BytesReceived`, `_BytesRejected`, `_RxQueueFull`, `_ReadsServiced`, `_WritesServiced`) with `Snapshot(&self) -> NodeStats` and `Reset(&self)`.
  - Embed `AtomicNodeStats` directly in `CrewNode` and `Arc<AtomicNodeStats>` in `VMAdaptor`, removing `SpinMutex<NodeStats>` from `VMAdaptor`.
  - In `VMAdaptor`, increment `_BytesRejected` and `_RxQueueFull` when an MMIO `REG_TX_DATA` write occurs while `txQueue.IsFull()` or when link RX is full.
- **Pre-Resolve Peer References in `CrewHub` (`src/crew/hub.rs`)**:
  - Add `_Peers: SpinMutex<Buff<Arc<CrewNode>>>` (or `ArcSwap`-style snapshot) on `CrewNode`, updated whenever `CrewHub::AddNode` or `CrewHub::AddLink` is called.
  - In `CrewHub::HandleRequest`, iterate the node's resolved `Arc<CrewNode>` slice directly without locking `_Routes`, locking `_Nodes`, or allocating a `Buff<u32>`.
  - Read `_MessageCb` once before delivering a byte across peers.
  - Replace `.Traverse(...)` with `.Span(...)` in `FindNode` and `AddLink` so searches short-circuit immediately on match.

Step 1.3: Fast `CoroPorts` Construction (`src/crew/vm_runner.rs` & `src/crew/vm_adaptor.rs`)

- Add a bulk constructor `CoroPorts::FromSlice(&[u64])` (or fixed-array initializer) in `rube` and use it in `VmBus::Idle()`, `VmBus::Read()`, `VmBus::Write()`, and step 6 of `VMAdaptor::New`.

Phase 2: UI Rendering, Text Shaping & Async Loading (`fascia`)

Step 2.1: Single-Pass Label Measurement & Monotonic Depth Levels (`src/fascia/cask_labels.rs`, `src/fascia/cask_scene.rs`)

- Store the measured `[f32; 2]` label size on each `CaskNode` during `CaskScene::Layout` so `LabelPlan::PlanForUnfurled` reads the cached size from `unfurled.Nodes()` without locking `FONTS` or re-running `Buffer::set_text`.
- Choose `let shaping = if name.is_ascii() { Shaping::Basic } else { Shaping::Advanced };` in `LayoutScene` and `Raster`.
- Ensure `levels` in `cask_labels.rs` and `cask_scene.rs` is monotonically non-decreasing across depths (`levels[d] = levels[d].max(levels[d - 1])`) after populating node prefix counts.

Step 2.2: Waveform Pixel-Column Decimation & Caching (`src/fascia/waveform.rs`)

- In `WaveformLane::draw`, compute the pixel column `col = x.floor() as i32` for each transition in `changes`.
```

- When multiple transitions map to the same `col`, draw a single vertical min/max envelope segment at `col` rather than stroking $2N$ overlapping sub-pixel segments.
- Add `canvas::Cache` to `WaveformState` and invalidate it only inside `WaveformState::Update` when `_ViewStart`, `_ViewEnd`, or `_CursorTime` changes.

Step 2.3: Cask 2D Canvas Caching, Search Indexing & Reverse Hit-Testing (`src/fascia/cask_view.rs`)

- Add a `matched_indices: Vec<usize>` field to `Cask2dViewerState` updated in `Cask2dAction::Search` and `Recompute()`, removing `.to_lowercase()` string allocations from `Cask2dCanvasProgram::draw`.
- In `Cask2dCanvasProgram::update`, change `.iter().position(|n| n.ContainsPoint(world_x, world_y))` to `.iter().rposition(|n| n.ContainsPoint(world_x, world_y))` so clicking nested boxes selects the innermost node under the cursor.
- Cache static background and box geometry in a `canvas::Cache`.

Step 2.4: Persistent Background `Atelier` Service (`src/fascia/geometry_load.rs`, `src/fascia/examples.rs`)

- Replace the per-job `Atelier::New(1)` creation inside `geometry_load::Queue()` and `examples::Execute()` with a single shared background `Atelier` instance initialized via `OnceLock`.
- Move `.vcd` file reading + `ParseVcd` and `Cask2dLayout::FromUri` off the UI thread into `Task::perform` using the shared background pool.
- Store a shared `XplrRegistry` inside `ExplorerState` instead of calling `XplrRegistry::New()` on every directory expansion.

Phase 3: Modularization & Code Simplification (`fascia`, `crew`, `drove`, `cove`)

Step 3.1: Consolidate Tab State into `TabDocument` (`src/fascia/tabs.rs`, `src/fascia/app.rs`)

- Define `TabDocument`:

```
pub enum TabDocument {
    Welcome,
    Settings,
    Editor(text_editor::Content),
    Waveform(WaveformState),
    Geometry(GeometryViewerState),
    Cask2d(Cask2dViewerState),
    PythonConsole(PythonConsoleState),
}
```

- Store `TabDocument` alongside `TabItem` inside `TabManager` (or a single `BTreeMap<TabId, TabDocument>`).
- Remove `open_editors`, `open_waveforms`, `open_consoles`, `open_cask2d_views`, and `_GeometryViews` from `AppState`.
- Implement `Drop` on `PythonConsoleState` to call `self.cancel()`, guaranteeing cleanup whether a tab is closed individually or via `CloseAllTabs`.
- Fix cursor reporting in `AppMessage::EditorAction`:

```
let (line, col) = editor.cursor_position();
self.status_info.line = line + 1;
self.status_info.column = col + 1;
```

Step 3.2: Split `src/fascia/app.rs` into Focused View Submodules

- Extract `view_welcome`, `view_settings`, `view_search_sidebar`, and `view_editor` out of `src/fascia/app.rs` into `src/fascia/views/{welcome,settings,search,editor}.rs` so `app.rs` strictly handles top-level state transitions and shell composition.

Step 3.3: Clean Up `crew` API Surface & `ProtocolMessage` Alignment (`src/crew/`)

- Remove redundant `snake_case` forwarding methods from `CrewHub`, `CrewNode`, and `ProtocolMessage`, keeping standard `PascalCase` methods to match the rest of the Trellis codebase.
- Change `ProtocolMessage` from `#[repr(C, packed)]` to `#[repr(C)]`, add `pub const WIRE_SIZE: usize = 24;`, and update `ProtocolStructure` test assertions to verify `ProtocolMessage::WIRE_SIZE == 24` and `ToLeBytes().len() == 24`.

Step 3.4: Enhance `cove` Assertions & Runner Filtering (`src/cove/`)

- Update `jeeves_assert!`, `jeeves_assert_eq!`, and `jeeves_assert_ne!` in `src/cove/jeeves.rs` so optional failure messages `$(, $msg:expr)?` are formatted into `eprintln!` when an assertion fails.
- In `src/cove/runner.rs`, evaluate `kind_matches` before formatting `full_name`, and pre-compute `let filter_lower = filter_str.to_lowercase();` once before iterating `inventory::iter::<TestCase>`.

Step 3.5: Separate `drove` Host Wrappers from SPIR-V Device Crate (`src/drove/`)

- Extract shared shader entry-point name constants into a lightweight `no_std` module shared by both `src/drove/` (host) and `src/drove/src/` (SPIR-V device crate) so entry-point names (`"compute::double_cs"`, `"vs_quad"`, `"vs_mesh"`) are verified from a single source of truth.

4. Verification & Benchmark Checklist

1. **Functional & Regression Suite**:
   - Run `cargo test --features tests` and Cove runner (`cargo run -- -test -v 2`) to verify `Cove`, `Crew`, and `Fascia` suites pass with zero assertion failures.
2. **Allocation & Latency Benchmark (`rube_perf`)**:
   - Run `cargo run --release --example rube_perf -- 4096 100` before and after Phase 1 to verify zero steady-state allocations per drive (`allocations/drive=0`, `allocated_bytes/drive=0`) and compare serial vs. 4-worker parallel latency.
3. **UI Memory Stability**:
   - Launch Fascia (`cargo run --features tests -- -e Fascia::Example`), move the cursor continuously across the menubar and 2D/3D viewports for 60 seconds, and confirm process resident memory remains flat. '@ | Set-Content -Path "C:\Work\Lapwing\Trellis\Wiki\Implementation_Plan.md" -Encoding utf8

```
---

## 3. Optional Part 2: Core Engine Internals (`rube`, `silo`, `stalks`, `heist`, `fenst`, `fleck`, `swarm`, `symph`)

Because `trellis_dump.txt` was cut off alphabetically inside `src\fascia\_tests.rs`[cite: 1], the lower-level engine

If you would like me to audit those core modules (e.g., `SimEngine` parallel scheduling in `rube`, `SpinMutex`/`Coro

```powershell
Get-ChildItem -Path .\Cargo.toml,.\build.rs,.\src\lib.rs,.\src\main.rs,.\src\fenst,.\src\fleck,.\src\heist,.\src\rub
  Where-Object { $_.Name -notmatch '^_tests\.rs$' } |
  ForEach-Object { "=== FILE: $($_.FullName) ==="; Get-Content $_.FullName; "" } |
  Out-File -FilePath .\trellis_core_dump.txt -Encoding utf8
```

```